# Indian Institute of Technology Bombay

Embedded Systems Design (EE712), 2026

---

# Course Project Report

---

## Real-Time Driver Assistance System using Intelligent Decision Making based on Depth Calculation and Image Segmentation


**Team ID:** TEAM_05

| Name | Roll No. |
|---|---|
| Amol Pagare | 22B3971 |
| Arnav Agarwal | 22B3917 |
| Neil George | 22B3963 |
| Mitul Tandon | 22B0305 |
| Tanish Raghute | 22B3974 |

Instructor:
Prof. Laxmeesha Somappa


May 4, 2026

# 0. Contents

**1 Introduction and Motivation** **2**

**2 Revised System Architecture** **2**

**3 Dual Camera Interfacing** **3**
3.1 Problem Statement . . . 3
3.2 Laptop-as-Arbiter Solution . . . 3

**4 Segmentation Model: SimpleFCN** **3**
4.1 Model Selection Rationale . . . 3
4.2 Architecture . . . 3
4.3 Training and Quantisation . . . 4

**5 Segmentation Accelerator** **4**
5.1 HLS Synthesis . . . 4
5.2 DPU vs. Custom IP Decision . . . 4

**6 PYNQ-Z2 Resource Constraints and Utilisation** **5**
6.1 BRAM Bottleneck . . . 5
6.2 Post-Implementation Utilisation Summary . . . 5

**7 Overlay and Video Pipeline Integration** **5**
7.1 Modifying the Base Overlay . . . 5
7.2 Video Output and HDMI Integration . . . 6

**8 Results and Demonstration** **6**
8.1 Functional Verification . . . 6
8.2 Performance Observations . . . 6
8.3 Known Limitations . . . 6

**9 Individual Contributions** **7**

**10 Conclusion** **7**

## 1. Introduction and Motivation

Road accidents remain one of the leading causes of preventable fatalities globally, claiming approximately 1.35 million lives annually according to the World Health Organization (WHO). Advanced Driver Assistance Systems (ADAS) represent a key technological response: by providing real-time situational awareness—lane detection, obstacle identification, and depth estimation—these systems can significantly reduce the cognitive burden on drivers and intervene before collisions occur.

Modern embedded platforms such as the PYNQ-Z2 development board, which couples an ARM Cortex-A9 Processing System (PS) with a Xilinx Zynq Programmable Logic (PL) fabric, offer a compelling target for low-cost, low-power ADAS prototyping. The tight hardware/software co-design enabled by the PS+PL architecture allows computationally intensive tasks (e.g., semantic segmentation) to be offloaded to dedicated accelerators on the PL, while orchestration and lighter algorithms run on the PS—achieving real-time responsiveness within strict resource budgets.

This report documents the complete implementation journey of the EE712 course project: from initial proposal through hardware bring-up, model design, accelerator synthesis, system integration, and final validation.

## 2. Revised System Architecture

The originally proposed architecture, presented in the Course Project Proposal (March 8, 2026), envisioned a symmetric pipeline: dual USB webcams feeding DDR RAM via PS DMA, with parallel depth estimation (PS) and segmentation (PL/DPU) pipelines merging into a decision system with HDMI output.

During implementation, several constraints necessitated design modifications—most significantly in camera interfacing and segmentation model selection. The revised high-level data flow is shown in Figure 1.

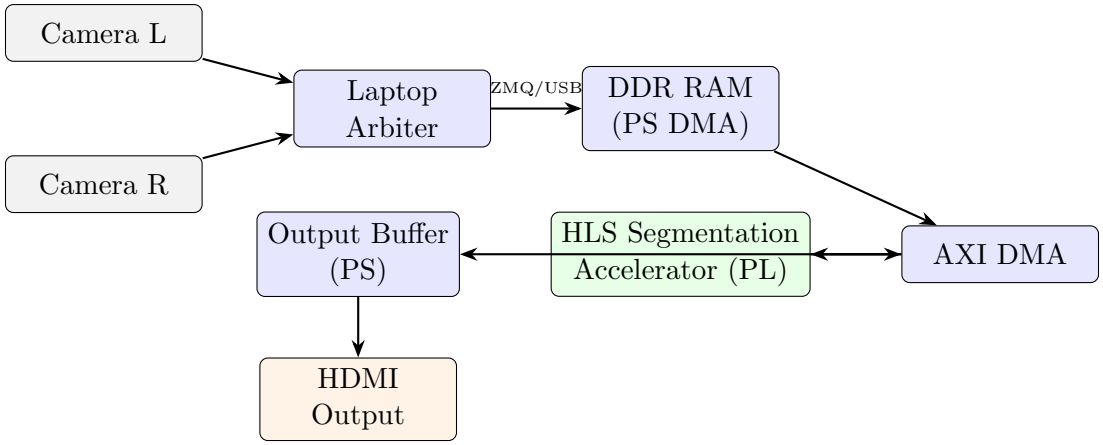



Figure 1: Revised ADAS system data flow. The laptop acts as a frame-multiplexing arbiter for dual-camera input; the HLS-synthesized segmentation accelerator runs on the PL.

# 3. Dual Camera Interfacing

## 3.1 Problem Statement

The original plan called for two identical USB webcams connected to the PYNQ-Z2 via a USB hub. During bring-up, two critical issues were encountered:

1. **Insufficient hub power:** A passive USB hub could not supply sufficient current to power both cameras simultaneously, causing one or both cameras to drop out unpredictably.
2. **Cost of powered hub:** A powered USB hub that could reliably drive dual webcams was judged prohibitively expensive relative to the project budget.

## 3.2 Laptop-as-Arbiter Solution

The team devised a software-level workaround: a host laptop acts as a frame-multiplexing arbiter. Each camera is directly connected to the laptop. A lightweight Python server (using ZeroMQ for efficient socket communication) captures one frame from Camera L, then one frame from Camera R, alternating at each capture tick. These interleaved frames are streamed over the local router/network to the PYNQ-Z2 PS.

This approach eliminated the hardware power issue entirely and required no additional component procurement. The primary trade-off is a slight increase in end-to-end latency introduced by the network hop, which was deemed acceptable for the demonstration setting.

# 4. Segmentation Model: SimpleFCN

## 4.1 Model Selection Rationale

The original proposal targeted ENet (352k parameters) deployed via the Xilinx DPU. Post-proposal analysis revealed a fundamental BRAM bottleneck: the PYNQ-Z2 provides only **630 KB** of on-chip block RAM, whereas ENet's weights alone require approximately **2.8 Mb**—over four times the available capacity, not counting activation buffers.

A custom model, termed *SimpleFCN*, was designed from scratch to fit within the resource envelope while preserving the segmentation objective.

## 4.2 Architecture

SimpleFCN is a lightweight Fully Convolutional Network (FCN) with the following profile:

Table 1: SimpleFCN deployment profile

| **Property** | **Value** |
|---|---|
| Total Parameters | 11,116 |
| Quantization | INT8 (8-bit integer post-training quant.) |
| Weight Memory Footprint | 10.9 KB |
| Input Resolution | 128×128 pixels |
| Output Classes | 4 (Road, Obstacle, Structure, Background) |

The network is a straightforward encoder–decoder: a series of depthwise-separable convolutions in the encoder halve the spatial resolution while increasing channel depth, followed by bilinear upsampling in the decoder to restore the 128×128 output resolution. Batch normalisation is fused into weights at export time to eliminate inference-time overhead.

### 4.3 Training and Quantisation

The model was trained in PyTorch on a publicly available urban driving segmentation dataset, then quantised to INT8 using post-training quantisation. INT8 quantisation provides a 4× reduction in weight storage (from ˜43 KB at FP32 to 10.9 KB) and maps naturally onto the DSP arithmetic available on the Zynq fabric.

## 5. Segmentation Accelerator

### 5.1 HLS Synthesis

The quantised SimpleFCN inference kernel was implemented as a custom Programmable Logic IP using **Vitis HLS**. The HLS design:

- Takes a 128×128 INT8 input frame via AXI-Stream.
- Computes convolution layers using fully-pipelined MAC operations.
- Produces a 128×128 pixel-class output map via AXI-Stream.
- Is integrated into the block design via **AXI DMA** for PS↔PL data transfer.

Vitis HLS directives (`PIPELINE`, `DATAFLOW`, `ARRAY_PARTITION`) were applied to maximise throughput while keeping resource utilisation within budget.

### 5.2 DPU vs. Custom IP Decision

Initially, integration via the Xilinx DPU (via Brevitas and FINN toolchains) was explored by Neil George. However, the DPU toolchain introduced significant version-compatibility issues between Vitis AI, PYNQ overlay packages, and the board support package. Given the time constraints, the team pivoted to a hand-written HLS kernel, which offered direct control over resource usage and eliminated the toolchain dependency chain.

# 6. PYNQ-Z2 Resource Constraints and Utilisation

## 6.1 BRAM Bottleneck

The PYNQ-Z2 (based on the Xilinx XC7Z020) provides a total of **630 KB** of on-chip BRAM across 140 RAMB36 tiles. This was the primary constraint driving the switch from ENet to SimpleFCN, as summarised in Table 2.

Table 2: Weight memory comparison

| Model | Params | Memory (FP32) | Memory (INT8) |
|---|---|---|---|
| ENet (original target) | 352,000 | 1.4 MB | 352 KB |
| SimpleFCN (deployed) | 11,116 | 43.4 KB | 10.9 KB |

Note that raw weight size understates actual BRAM demand: activation maps, line buffers, and DMA FIFOs further consume BRAM. The decision to use SimpleFCN provided a comfortable margin.

## 6.2 Post-Implementation Utilisation Summary

Post-implementation resource utilisation for the complete system (base overlay + segmentation accelerator + HDMI modules + AXI interconnect) was within acceptable limits for the XC7Z020 device. Key resource categories are summarised in Table 3.

Table 3: Post-implementation utilisation summary (XC7Z020)

| Resource | Used | Available | Utilisation (%) |
|---|---|---|---|
| LUT | 9006 | 53,200 | 16.93 |
| LUTRAM | 162 | 17,400 | 0.93 |
| FF | 11145 | 106,400 | 10.47 |
| BRAM (36K) | 121.5 | 140 | 86.79 |
| DSP | 29 | 220 | 13.18 |

# 7. Overlay and Video Pipeline Integration

## 7.1 Modifying the Base Overlay

The PYNQ-Z2 ships with a "base" Vivado overlay that includes pre-built AXI DMA, HDMI input/output, and audio controllers. Integrating the custom HLS segmentation IP required modifying this base overlay:

1. The HLS-exported IP was packaged and added to the Vivado IP repository.
2. An AXI DMA instance was wired to the segmentation IP's AXI-Stream ports (MM2S for input frames, S2MM for segmented output).
3. AXI Interconnect and AXI SmartConnect blocks were updated to route PS↔DMA↔Seg-IP traffic without address conflicts.

4. The modified block design was synthesised, implemented, and exported as a new `.bit` + `.hwh` overlay pair for PYNQ.

### 7.2 Video Output and HDMI Integration

Displaying segmented frames over HDMI required integrating the Xilinx Video Processing Subsystem (VPS) and AXI4-Stream to Video Out IP cores into the overlay. This introduced additional complexity:

- **Clock domain management:** The pixel clock (148.5 MHz for 1080p) and the PL fabric clock (100 MHz) required careful FIFO buffering at clock-domain crossings.
- **Version incompatibilities:** Several Vivado IP versions in the base overlay conflicted with the target Vivado/PYNQ version, requiring manual IP upgrade steps.
- **Frame pacing:** The AXI DMA burst lengths and line-buffer depths were tuned to avoid underruns in the video pipeline.

A wireless feed (over the local router) was used as a fallback display path for functional demonstration when HDMI integration was being debugged—the segmented output was streamed and rendered on the host laptop.

## 8. Results and Demonstration

### 8.1 Functional Verification

The deployed system successfully performed semantic segmentation at the PYNQ-Z2, classifying pixels into four categories: Road, Obstacle, Structure, and Background. Key achieved results are:

- **Model size:** 10.9 KB (INT8 weights), comfortably within the 630 KB BRAM budget.
- **Wireless streaming:** Real-time segmented output streamed over Wi-Fi to the host laptop, confirming end-to-end pipeline functionality.
- **Dual-camera:** Both cameras successfully interfaced through the laptop-arbiter architecture, delivering alternating left/right frames to the PS.

### 8.2 Performance Observations

Owing to the significantly smaller model (11k vs. 352k parameters), the computational load on the HLS accelerator was substantially reduced compared to the original ENet target. This translated to fast per-frame inference, though precise FPS measurements are deferred to the attached project repository and demo video.

### 8.3 Known Limitations

- The depth estimation pipeline was de-scoped from the final deliverable due to time constraints after dual-camera interfacing and BRAM bottleneck resolution consumed significant sprint time.

- The rule-based decision-making module (braking/speed commands) was not integrated in the final demo; the pipeline terminates at segmented frame output.
- HDMI output required overlay modifications that introduced version-compatibility issues; the wireless feed served as the primary demo path.

## 9. Individual Contributions

Table 4: Team member contributions

| Member | Contributions |
|---|---|
| Amol Pagare | Dual camera interfacing; Vitis HLS RTL synthesis for segmentation model; custom PL IP integration; resource utilisation analysis. |
| Arnav Agarwal | Segmentation model training and architecture design (SimpleFCN); INT8 post-training quantisation; resource utilisation analysis; Vitis HLS synthesis support. |
| Neil George | Explored DPU-based integration using Brevitas and FINN toolchains; documented compatibility issues and informed the pivot to HLS. |
| Mitul Tandon | Segmentation model comparison and selection; optimisation of model for BRAM-constrained deployment. |
| Tanish Raghute | Dual camera interfacing; modifications to base overlay for HDMI video output pipeline. |

## 10. Conclusion

This project demonstrated a real-time semantic segmentation pipeline on the PYNQ-Z2 embedded platform within the strict BRAM and power constraints of the XC7Z020 SoC. The key engineering contributions were:

1. **Resource-aware model design:** SimpleFCN (11k parameters, INT8) was purpose-built to fit within the 630 KB BRAM budget, a direct response to the ENet infeasibility discovered post-proposal.
2. **Hardware-in-the-loop HLS accelerator:** The inference kernel was synthesised directly via Vitis HLS and integrated as a custom PL IP, avoiding DPU toolchain compatibility issues.
3. **Creative I/O solution:** The laptop-as-arbiter approach for dual-camera interfacing resolved power-budget limitations without additional cost.

Future work would include integrating the stereo depth estimation pipeline (originally proposed), implementing the rule-based decision system, and validating HDMI output at full 1080p resolution. Quantisation-aware training and further model pruning could also enable higher accuracy at comparable parameter counts.